

Comunicaciones Digitales.

Práctica 1.

Luis Gerardo Anguiano Ponce. 3301850.

Abstracto—Aquí se presenta el análisis de datos obtenidos a partir de phyphox, utilizando los entornos de programación de MATLAB y Python.

I. INTRODUCCIÓN.

En el campo de la ingeniería en telecomunicaciones la adquisición, procesamiento y análisis de datos son fundamentales para el desarrollo de sistemas de comunicación. En esta práctica de laboratorio se utilizaron los sensores integrados en un dispositivo móvil, mediante la aplicación phyphox, como fuente principal de datos experimentales. Esta herramienta permite registrar, visualizar y exportar mediciones físicas, como aceleración, presión o velocidad angular.

El objetivo de esta práctica fue registrar ciertos parámetros físicos mediante phyphox y posteriormente analizar los datos obtenidos utilizando dos entornos de programación: MATLAB, ampliamente usado en el análisis numérico y procesamiento de señales, y Python, un lenguaje de programación versátil y popular en el análisis de datos científicos.

Finalmente, se realizó una comparación entre los resultados procesados por ambas herramientas con el fin de evaluar su precisión, facilidad de uso y efectividad en el tratamiento de los datos obtenidos.

II. DESARROLLO.

2.1. PHYPHOX

Para esta parte, se eligieron 3 experimentos de phyphox.org. Se llevaron a cabo las indicaciones para la toma de mediciones, y se exportaron los datos en formato .csv para el procesamiento.

2.1.1. EXPERIMENTO 1. ACELERACIÓN.

Este experimento consistió en una pequeña prueba de la aplicación. Utilizando la opción “aceleración (sin g)” y desplazando el dispositivo móvil en distintas direcciones se generan gráficas de aceleración en los 3 ejes.

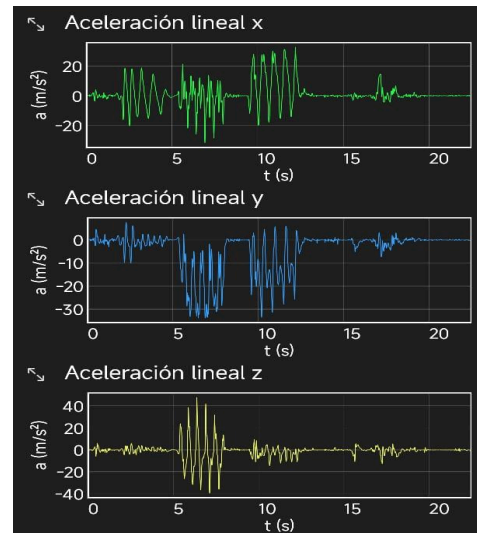


Figura 1. Resultado del experimento 1 en phyphox.

2.1.2. EXPERIMENTO 2. ELEVADOR.

El experimento 2 consiste en la obtención de la velocidad y altura alcanzadas en un elevador, se requiere que el dispositivo cuente con un barómetro o sensor de presión para el cálculo de la altura y la velocidad de movimiento del ascensor. Se genera una gráfica de la presión contra tiempo medida, y gráficas con la altura y velocidad calculadas.

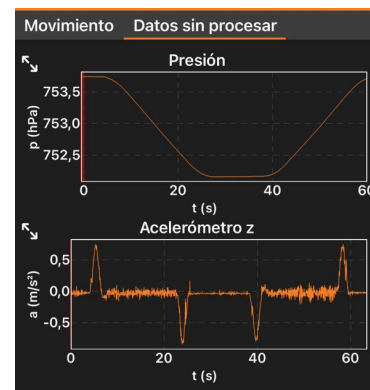


Figura 2. Resultado del experimento 2 en phyphox. Información medida por el sensor.

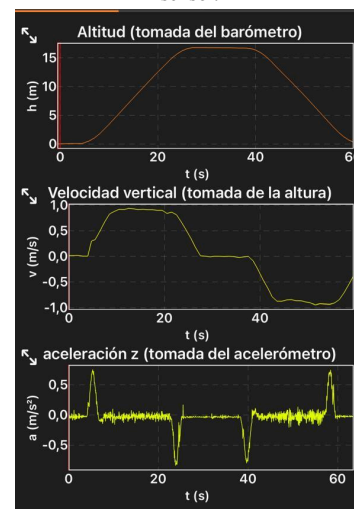


Figura 3. Resultado del experimento 2 en phyphox. Información calculada por phyphox.

2.1.3. EXPERIMENTO 3. RADAR.

Este experimento consistió en convertir el móvil en un radar, se requiere de un celular con micrófono y bocina, además de un ambiente tranquilo y silencioso. El sensor mide el retraso desde la emisión de un chirrido de frecuencia conocida hasta su detección, a partir de lo cual se puede obtener una aproximación de la distancia del celular al objeto reflector fijando la velocidad del sonido, o de la velocidad del sonido fijando una distancia al objeto reflector.



Figuras 4 y 5. Desarrollo del experimento 3.

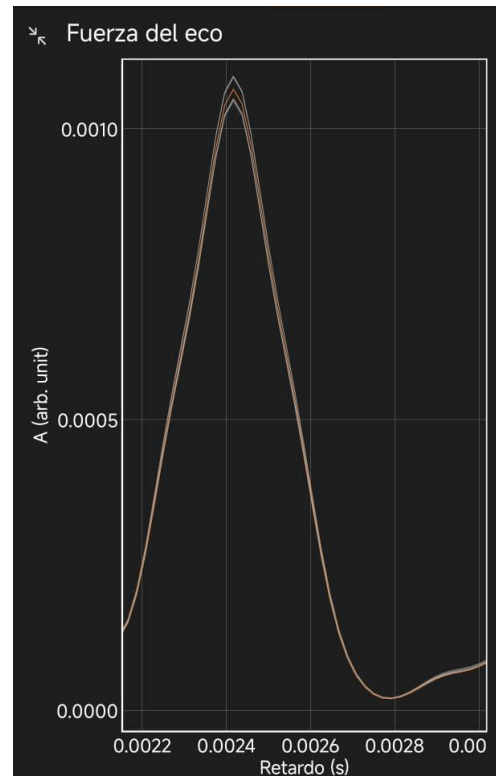


Figura 6. Resultado del experimento 3 en phyphox. Retardo medido por el sensor.

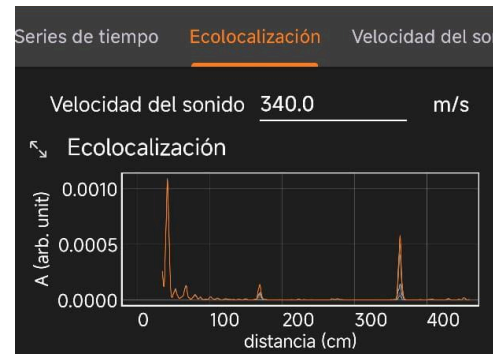


Figura 7. Resultado del experimento 3 en phyphox. Distancia calculada por la app tomando como referencia la velocidad del sonido de 340 m/s.



Figura 8. Resultado del experimento 3 en phyphox. Velocidad del sonido calculada por la app tomando como referencia la distancia al objeto de 50 cm.

2.2. MATLAB

Para el análisis de datos en MATLAB, se recurrió a los archivos .csv exportados directamente de phyphox. Una vez

en el entorno de programación, se hace la importación de la información dentro del workspace para su procesamiento. Para los tres experimentos se realizó un código muy similar, el cual consiste en asignar los datos de cada columna a una variable y posteriormente realizar un subplot para cada una de las magnitudes deseadas.

2.2.1. EXPERIMENTO 1.

```

1  t = RawData.Time_s_ ;
2  x = RawData.LinearAccelerationX_m_s_2;
3  y = RawData.LinearAccelerationY_m_s_2;
4  z = RawData.LinearAccelerationZ_m_s_2;
5  aabs = RawData.AbsoluteAcceleration_m_s_2;
6
7  g = figure;
8  subplot(4,1,1);
9  plot(t,x,'Color','#AF172E');
10 xlabel('Time [s]');
11 ylabel('Acc.X [m/s^2]');
12 grid on
13
14 subplot(4,1,2);
15 plot(t,y);
16 xlabel('Time [s]');
17 ylabel('Acc.Y [m/s^2]');
18 grid on
19
20 subplot(4,1,3);
21 plot(t,z,'Color','#0DA5D0');
22 xlabel('Time [s]');
23 ylabel('Acc.Z [m/s^2]');
24 grid on
25
26 subplot(4,1,4);
27 plot(t,aabs,'Color','#D16A12');
28 xlabel('Time [s]');
29 ylabel('Abs.Acc. [m/s^2]');
30 grid on
31
32 saveas(g, 'GraficaExp1.png');

```

Figura 9. Código en MATLAB para experimento 1.

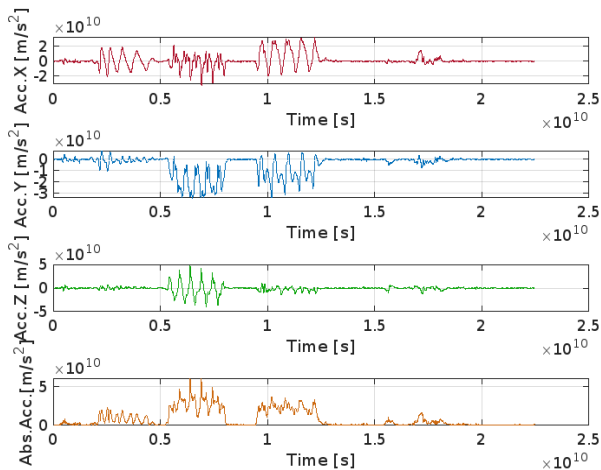


Figura 10. Resultado en MATLAB para experimento 1.

2.2.2 EXPERIMENTO 2.

```

1  th=PressureAndVelocity.Time_s_;
2  h=PressureAndVelocity.Altitude_m_;
3  p=PressureAndVelocity.Pressure_hPa_;
4
5  tv=PressureAndVelocity.Time_velocity__s_;
6  yv=PressureAndVelocity.Velocity_m_s_;
7
8  ta=Acceleration.Time_s_;
9  A=Acceleration.Acceleration_m_s_2;
10
11
12 f = figure;
13 subplot(4,1,1);
14 plot(th,h);
15 xlabel('Time [s]');
16 ylabel('h [m]');
17 grid on
18
19 subplot(4,1,2);
20 plot(tv,yv);
21 xlabel('Time [s]');
22 ylabel('Vel.Vert.[m/s]');
23 grid on
24
25 subplot(4,1,3);
26 plot(ta,A);
27 xlabel('Time [s]');
28 ylabel('Acc.Z [m/s^2]');
29 grid on
30
31 subplot(4,1,4);
32 plot(th,p);
33 xlabel('Time [s]');
34 ylabel('P [hPa]');
35 grid on
36
37 saveas(f, 'GraficaExp2.png');

```

Figura 11. Código en MATLAB para experimento 2.

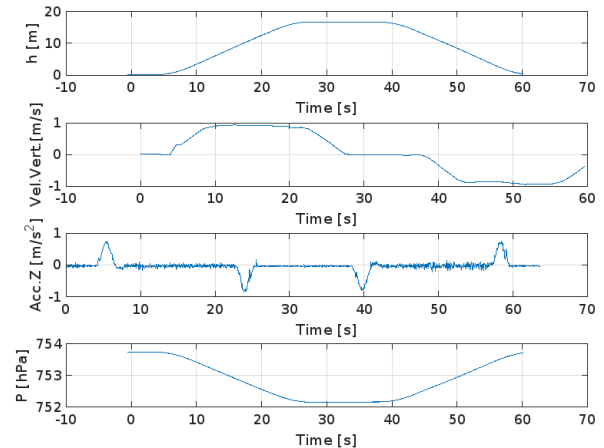


Figura 10. Resultado en MATLAB para experimento 2

2.2.3. EXPERIMENTO 3.

```

Practica1Exp2.m x Practica1Exp1.m x PrácticaExp3.m x +
MATLAB Drive/Com_Digit_P1/Exp3/PrácticaExp3.m
1 Distancia = EchoLocation.Distance_cm_;
2 A = EchoLocation.Crosscorrelation_a_u_;
3
4 Delay = Time.Delay_s_;
5 ADelay = Time.Crosscorrelation_a_u_;
6
7 Velocidad = SpeedOfSound.SpeedOfSound_m_s_;
8 Avel = SpeedOfSound.Crosscorrelation_a_u_;
9
10 h = figure;
11 subplot(3,1,1);
12 plot(Delay,ADelay);
13 xlabel("Retraso [s]");
14 ylabel("Amplitud [u.arb]");
15 grid on;
16
17 subplot(3,1,2);
18 plot(D,A);
19 xlabel("Distancia [cm]");
20 ylabel("Amplitud [u.arb]");
21 grid on;
22
23 subplot(3,1,3);
24 plot(Velocidad,Avel);
25 xlabel("Velocidad sonido [m/s]");
26 ylabel("Amplitud [u.arb]");
27 grid on;
28
29 saveas(h,'GraficaExp3.png');

```

Figura 11. Código en MATLAB para experimento 1.

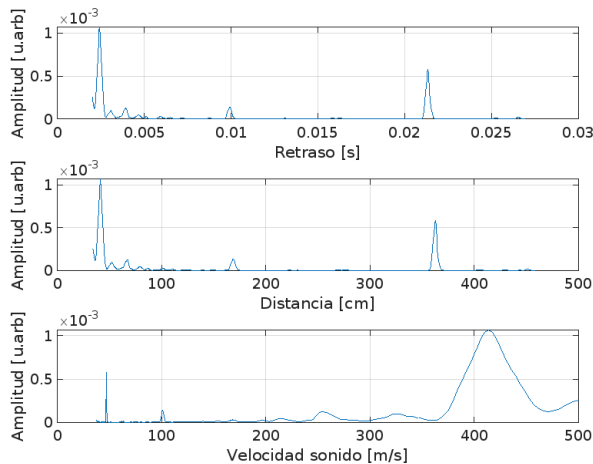


Figura 12. Resultado en MATLAB para experimento 3.

2.3. PYTHON

En esta parte de la práctica se realizó el mismo proceso de análisis de datos, pero ahora en un entorno de programación de python y se optó por utilizar google colab. Para el análisis de datos en python, se necesita de la librería “pandas” y para la visualización de los resultados la librería “matplotlib”.

Para realizar la importación de los archivos .csv al entorno de google colab, se utilizó el siguiente código:

```

from google.colab import files
uploaded = files.upload()
for fn in uploaded.keys():
    print(f'User uploaded file "{fn}" with length {len(uploaded[fn])} bytes')

```

Figura 13. Código para subir archivos a colab.

2.3.1. EXPERIMENTO 1.

```

import pandas as pd
import matplotlib.pyplot as plt

# Cargar datos desde archivos CSV
pressure_and_velocity = pd.read_csv('Pressure_and_velocity.csv', delimiter='\\t', header=0, na_values='')
acceleration = pd.read_csv('Acceleration.csv', delimiter='\\t', header=0, na_values='')

# Extraer columnas
th = pressure_and_velocity['Time (s)']
h = pressure_and_velocity['Altitude (m)']
p = pressure_and_velocity['Pressure (hPa)']

tv = pressure_and_velocity['Time (velocity) (s)']
yv = pressure_and_velocity['Velocity (m/s)']

ta = acceleration['Time (s)']
A = acceleration['Acceleration (m/s^2)']

# Convert relevant columns to numeric, coercing errors
th = pd.to_numeric(th, errors='coerce')
h = pd.to_numeric(h, errors='coerce')
p = pd.to_numeric(p, errors='coerce')
tv = pd.to_numeric(tv, errors='coerce')
yv = pd.to_numeric(yv, errors='coerce')
ta = pd.to_numeric(ta, errors='coerce')
A = pd.to_numeric(A, errors='coerce')

# Crear figura y subplots
fig, axs = plt.subplots(4, 1, figsize=(10, 12))

axs[0].plot(th, h, color='AF172E')
axs[0].set_xlabel('Time [s]')
axs[0].set_ylabel('h [m]')
axs[0].grid(True)

axs[1].plot(tv, yv, color='AF172E')
axs[1].set_xlabel('Time [s]')
axs[1].set_ylabel('Vel.Vert. [m/s]')
axs[1].grid(True)

axs[2].plot(ta, A, color='0D458D')
axs[2].set_xlabel('Time [s]')
axs[2].set_ylabel('Acc.Z [m/s^2]')
axs[2].grid(True)

axs[3].plot(th, p, color='0D458D')
axs[3].set_xlabel('Time [s]')
axs[3].set_ylabel('P [hPa]')
axs[3].grid(True)

# Ajustar espacios
plt.tight_layout()

# Guardar figura
plt.savefig('Grafica2.png')
plt.show()

```

Figura 14. Código en Python para experimento 1.

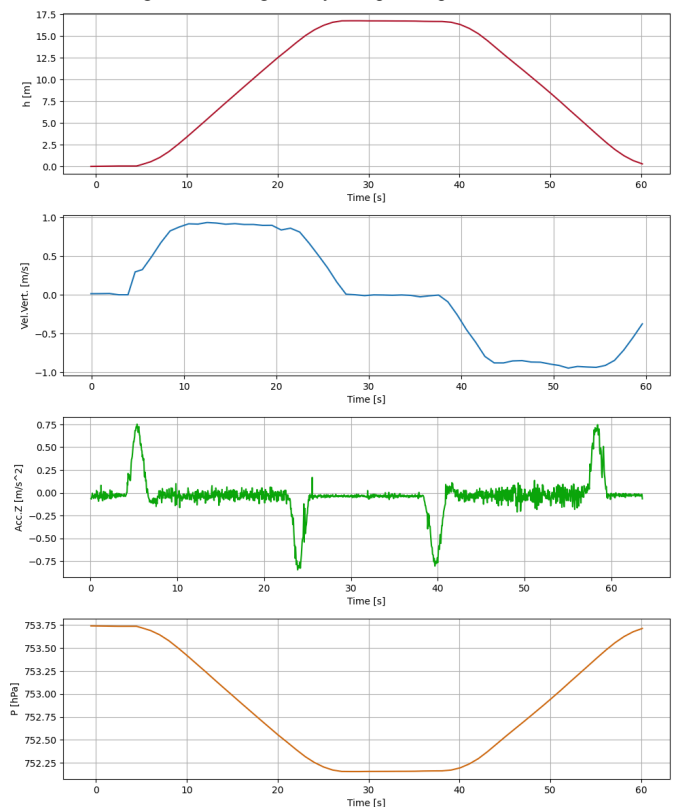


Figura 15. Resultado de Python para experimento 1.

2.3.2. EXPERIMENTO 2.

```
[ ] import pandas as pd
import matplotlib.pyplot as plt
# Cargar datos desde archivos CSV
accel = pd.read_csv('Raw Data.csv', delimiter='\t', header=0, na_values='', decimal=',')

# Extraer columnas
t = accel['Time (s)']
x = accel['Linear Acceleration x (m/s^2)']
y = accel['Linear Acceleration y (m/s^2)']
z = accel['Linear Acceleration z (m/s^2)']
aabs = accel['Absolute acceleration (m/s^2)']

# Convert relevant columns to numeric, coercing errors
t = pd.to_numeric(t, errors='coerce')
x = pd.to_numeric(x, errors='coerce')
y = pd.to_numeric(y, errors='coerce')
z = pd.to_numeric(z, errors='coerce')
aabs = pd.to_numeric(aabs, errors='coerce')

# Crear figura y subplots
fig, axs = plt.subplots(4, 1, figsize=(10, 12))

axs[0].plot(t, x, color='#AF172E')
axs[0].set_xlabel('Time [s]')
axs[0].set_ylabel('Acc.X [m/s^2]')
axs[0].grid(True)

axs[1].plot(t, y, color='#00A58B')
axs[1].set_xlabel('Time [s]')
axs[1].set_ylabel('Acc.Y [m/s^2]')
axs[1].grid(True)

axs[2].plot(t, z, color='#00A58B')
axs[2].set_xlabel('Time [s]')
axs[2].set_ylabel('Acc.Z [m/s^2]')
axs[2].grid(True)

axs[3].plot(t, aabs, color='#D01A12')
axs[3].set_xlabel('Time [s]')
axs[3].set_ylabel('Abs.Acc. [m/s^2]')
axs[3].grid(True)

# Ajustar espacios
plt.tight_layout()

# Guardar figura
plt.savefig('Grafica.png')
plt.show()
```

Figura 16. Código en Python para experimento 2

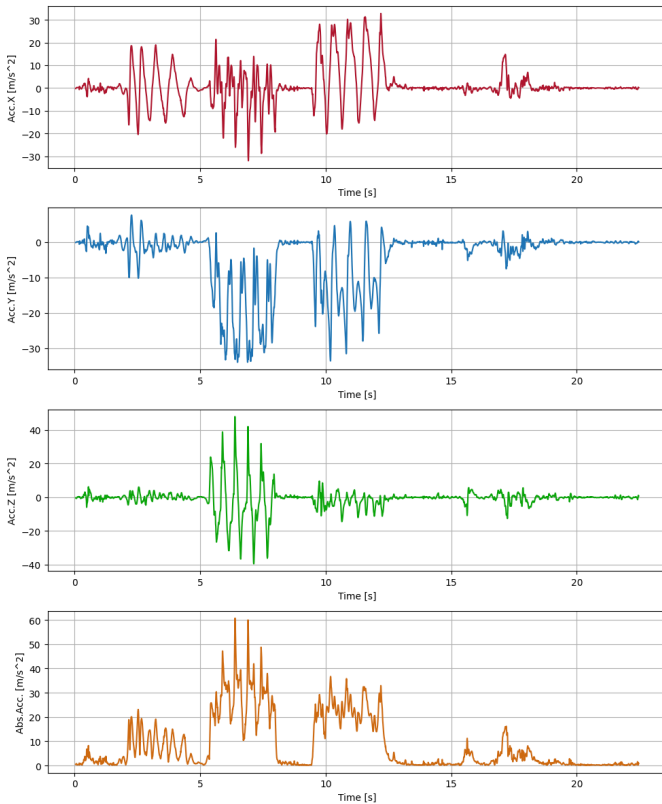


Figura 17. Resultado de Python para experimento 2.

2.3.3. EXPERIMENTO 3.

```
import pandas as pd
import matplotlib.pyplot as plt
import os

Echo = pd.read_csv('Echo location.csv', delimiter='\t', header=0, na_values='')
Speed = pd.read_csv('Speed of sound.csv', delimiter='\t', header=0, na_values='')
Time = pd.read_csv('Time.csv', delimiter='\t', header=0, na_values='')

# Extraer columnas
delay = Time['Delay (s)']
Adelay = Time['Crosscorrelation (a.u.)']

Distance = Echo['Distance (cm)']
ADistance = Echo['Crosscorrelation (a.u.)']

vel = Speed['Speed of sound (m/s)']
Avel = Speed['Crosscorrelation (a.u.)']

# Convert relevant columns to numeric, coercing errors
delay = pd.to_numeric(delay, errors='coerce')
Adelay = pd.to_numeric(Adelay, errors='coerce')
Distance = pd.to_numeric(Distance, errors='coerce')
ADistance = pd.to_numeric(ADistance, errors='coerce')
vel = pd.to_numeric(vel, errors='coerce')
Avel = pd.to_numeric(Avel, errors='coerce')

# Crear figura y subplots
fig, axs = plt.subplots(3, 1, figsize=(10, 12))

axs[0].plot(delay, Adelay, color='#AF172E')
axs[0].set_xlabel('Retardo [s]')
axs[0].set_ylabel('Amplitud [a.u.]')
axs[0].grid(True)

axs[1].plot(Distance, ADistance)
axs[1].set_xlabel('Distancia [cm]')
axs[1].set_ylabel('Amplitud [a.u.]')
axs[1].grid(True)

axs[2].plot(vel, Avel, color='#00A58B')
axs[2].set_xlabel('Velocidad del sonido [m/s]')
axs[2].set_ylabel('Amplitud [a.u.]')
axs[2].grid(True)

# Ajustar espacios
plt.tight_layout()

# Guardar figura
plt.savefig('grafica2.png')
plt.show()
```

Figura 18. Código de Python para experimento 3.

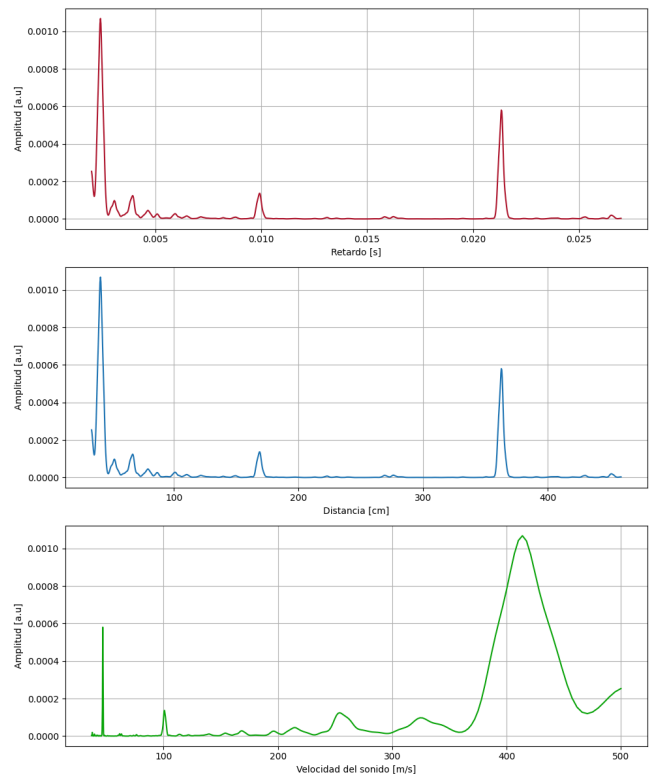


Figura 19. Resultado de Python para experimento 3.

III. ANÁLISIS DE RESULTADOS.

EXPERIMENTO 1:

El experimento 1 es muy sencillo, todas las mediciones las realiza el acelerómetro lineal. Para este experimento, el sensor contó con las siguientes características:

```
"linear_acceleration Name"      "linear_acc"
"linear_acceleration Vendor"    "mtk"
"linear_acceleration Range"     "39.226803"
"linear_acceleration Resolution" "0.0012"
"linear_acceleration MinDelay"  "5000"
"linear_acceleration MaxDelay"  "20000"
"linear_acceleration Power"     "0.001"
"linear_acceleration Version"   "1"
```

Figura 20. Especificaciones del acelerómetro lineal “Mediatek”.

EXPERIMENTO 2:

Phyphox utiliza el barómetro del dispositivo para medir el cambio de la presión atmosférica y a partir de esta medición calcula la diferencia de altura utilizando la fórmula barométrica.

$$P(h) = P_0 * \exp(-M * g * h / R * T)$$

Donde:

$P(h)$ = presión a una altura h .

P_0 = presión atmosférica en la altura cero.

M = Masa molar del aire (0.029 kg/mol).

g = aceleración de la gravedad (9.81 m/s²).

R = constante universal de los gases (8.314 J/mol*K).

T = temperatura (K)

De este modo, podemos calcular la altura y comprobar que el resultado haya sido correcto. Despejando h :

$$h = -M * g * R * T * \ln(P_0 / P(h))$$

$$P(h) \approx 752.20 \text{ hPa}$$

$$P_0 \approx 753.75 \text{ hPa}$$

$$M = 0.029 \text{ kg/mol}$$

$$g = 9.81 \text{ m/s}^2$$

$$R = 8.314 \text{ J/mol*K}$$

$$T \approx 16^\circ\text{C} = 289,15 \text{ K}$$

Reemplazando los valores: $h = 17.39 \text{ m}$. Valor muy aproximado a lo obtenido en la gráfica de phyphox.

EXPERIMENTO 3:

Phyphox utiliza la bocina del celular para emitir una señal de frecuencia conocida y el micrófono para detectar la señal cuando regresa reflejada por un objeto. El cálculo es:

$$\text{Velocidad: } V = \frac{d}{t} = \frac{2 * \text{distancia al objeto}}{t} = \frac{1 \text{ m}}{0.0024 \text{ s}}$$

$$V = 416.67 \text{ m/s}$$

Valor aproximado al obtenido en la gráfica de phyphox, tomando en cuenta la distancia del celular al objeto de 50 cm.

$$\text{Distancia: } D_{\text{total}} = v * t = 340 \text{ m/s} * 0.0024 \text{ s} = 0.816 \text{ m.}$$

$$D = D_{\text{total}} / 2 = 0.816 \text{ m} / 2 = 0.408 \text{ m.}$$

Valor aproximado al pico más alto de la gráfica “ecolocalización” entregada por phyphox.

ANÁLISIS. MATLAB/PYTHON:

En esta práctica, se utilizaron los entornos MATLAB y Python para analizar los datos generados. Ambos entornos produjeron resultados numéricamente muy similares, y la estructura de los códigos empleados fue comparable. En el caso de Python, se requirió el uso de librerías como pandas y matplotlib, que ofrecen una gran flexibilidad, aunque requieren conocimientos básicos de programación. Por otro lado, MATLAB integra de forma nativa funciones específicas para el análisis de datos, lo que puede facilitar el proceso en usuarios con menos experiencia en programación.

IV. CONCLUSIONES.

La aplicación Phyphox resultó ser una herramienta útil para la adquisición de datos experimentales, ya que no solo permite registrar información a través de los sensores del dispositivo, sino que también ofrece una interfaz gráfica inmediata para visualizar los resultados. Sin embargo, no todo experimento incluye representaciones gráficas predefinidas, lo que motiva el uso de herramientas externas para su procesamiento y análisis detallado.

MATLAB y Python demostraron ser buenos entornos para el procesamiento de datos. Ambos permiten realizar análisis de forma ágil y con resultados consistentes. Python, en particular, ofrece ventajas en entornos como Google Colab, donde se dispone de herramientas asistidas por inteligencia artificial que pueden acelerar el desarrollo del código. Por otro lado, MATLAB presenta una integración robusta de funciones matemáticas, aunque su disponibilidad puede estar limitada por su carácter de software con licencia.

V. REFERENCIAS.

[1] phyphox.org